HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# THỰC TẬP CƠ SỞ

## Weekly Report

## Đề tài: XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM

## Giai đoạn 2.1: Thu thập và làm sạch

| | |
|---|---|
| **Giảng viên** | **: Kim Ngọc Bách** |
| **Họ và tên sinh viên** | **: Nguyễn Văn Trường** |
| **Mã sinh viên** | **: B23DCCN870** |
| **Lớp** | **: D23CQCN02-B** |
| **Nhóm** | **: 11** |

***Hà Nội – 2026***

# Nội dung

I. Thu thập và xử lý dữ liệu ..... 3
1. Dataset được chọn: MovieLens 100K ..... 3
2. Lý do lựa chọn: ..... 3
3. Cấu trúc dataset ..... 3
I. Làm sạch dữ liệu ..... 4
1. Kiểm tra Missing Values ..... 4
2. Kiểm tra dữ liệu trùng lặp ..... 5
3. Kiểm tra giá trị bất thường ..... 5
4. Kiểm tra trùng lặp dữ liệu: ..... 5
5. Kiểm tra tính hợp lệ của UserId, MovieId: ..... 6
6. Kiểm tra tính nhất quán dữ liệu: ..... 6
Tổng kết: ..... 7
Phụ lục ..... 7
Đọc dữ liệu ..... 8
2. Kiểm tra chất lượng dữ liệu ..... 11

## I. Thu thập và xử lý dữ liệu

Lựa chọn dataset

Dữ liệu là thành phần quan trọng trong việc xây dựng hệ thống gợi ý. Chất lượng và cấu trúc của dữ liệu ảnh hưởng trực tiếp đến hiệu quả của mô hình.

1. Dataset được chọn: MovieLens 100K

MovieLens 100K là bộ dữ liệu chuẩn được cung cấp bởi GroupLens Research (Đại học Minnesota, Mỹ), chuyên dùng cho nghiên cứu về Recommender System.

Dataset được tải từ địa chỉ: https://grouplens.org/datasets/movielens/100k/

2. Lý do lựa chọn:

| Tiêu chí | Lý do |
|---|---|
| Kích thước phù hợp | 100, 000 ratings – đủ để train mô hình, ko quá nặng để chạy local |
| Cấu trúc rõ ràng | Có đủ thông tin: userId, movieId, rating, timestamp, thông tin phim và user |
| Chuẩn công nghiệp | Được sử dụng rộng rãi trong các bài báo nghiên cứu về CF và Matrix Factorization |
| Phù hợp bài toán | Dạng ma trận User-item thưa – phù hợp để minh họa sức mạnh của SVD |
| Miễn phí & có sẵn | Dữ liệu công khai, không cần xin phép |

3. Cấu trúc dataset

Dataset MovieLens 100K bao gồm 3 file chính:

| Thành phần | Mô tả |
|---|---|
| u.data | 943 users x 1682 movies – 100,000 ratings (userId, movieId, rating, timestamp) |
| u.item | Thông tin về 1,682 bộ phim: tiêu đề, năm phát hành, thể loại |

| | |
|---|---|
| u.user | Thông tin về 943 người dùng: tuổi, nghề nghiệp, giới tính |

I. Làm sạch dữ liệu

Trước khi tiến hành xây dựng mô hình, dữ liệu cần được kiểm tra và xử lý nhằm loại bỏ các giá trị không hợp lệ, dữ liệu thiếu hoặc trùng lặp. Quá trình làm sạch dữ liệu giúp đảm bảo chất lượng của dữ liệu đầu vào và nâng cao hiệu quả của mô hình.

Quá trình kiểm tra được thực hiện bằng ngôn ngữ python với thư viện Pandas.

1. Kiểm tra Missing Values

Dữ liệu được kiểm tra để phát hiện các giá trị thiếu (missing values) trong các trường quan trọng như userId, movieId, rating. Việc phát hiện và xử lý các giá trị thiếu giúp tránh lỗi trong quá trình xây dựng ma trận user-item và huấn luyện mô hình.

Câu lệnh kiểm tra: df.isnull().sum() để xác định số lượng giá trị thiếu trong từng bảng dữ liệu của dataset MovieLens.

Kết quả kiểm tra như sau:

```
────────────────────────────────────────
  3.1 MISSING VALUES
────────────────────────────────────────

  [Ratings (u.data)]  → Không có

  [Movies (u.item)]  →  CÓ 1 missing
release_date    1

  [Users (u.user)]  → Không có
```

Kết quả cho thấy:

- Bảng Ratings và Users không có giá trị thiếu.
- Bảng Movies có 1 giá trị thiếu tại thuộc tính release_date.

Để đảm bảo tính đầy đủ của dữ liệu, giá trị thiếu này được xử lý bằng cách thay thế giá trị “Unknown” trước khi tiếp tục các bước xử lý của dữ liệu

2. Kiểm tra dữ liệu trùng lặp

Dữ liệu trùng lặp có thể làm sai lệch kết quả huấn luyện mô hình. Vì vậy, các bản ghi trùng lặp được kiểm tra và loại bỏ. Mỗi bản ghi trong dataset cần đại diện cho một đánh giá duy nhất của một người dùng đối với một sản phẩm.

Sau khi kiểm tra, các bản ghi trùng lặp (nếu có) được loại bỏ để đảm bảo tính nhất quán của dữ liệu.

3. Kiểm tra  giá trị bất thường

Dataset cũng được kiểm tra để phát hiện các giá trị bất thường, chẳng hạn như:

- Điểm rating nằm ngoài khoảng cho phép:

Rating nằm ngoài khoảng khoảng hợp lệ [1,5]

Quá trình kiểm tra được thực hiện bằng cách xác định các giá trị rating không thuộc tập {1,2,3,4,5};

Kết quả kiểm tra như sau.

```
────────────────────────────────────────
3.2 KIỂM TRA RATING HỢP LỆ [1-5]
────────────────────────────────────────
Giá trị rating duy nhất: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
 Tất cả rating nằm trong khoảng [1, 5]
```

Kết quả kiểm tra cho thấy: tất cả các giá trị rating đều hợp lệ.

4. Kiểm tra trùng lặp dữ liệu:

Trong dataset hệ thống gợi ý, mỗi bản ghi trong bảng Ratings biểu diễn một người dùng đánh giá một bộ phim. Do đó, mỗi cặp (userId, movieId) chỉ nên xuất hiện một lần duy nhất.

Để đảm bảo tính nhất quán của dữ liệu, cần kiểm tra xem có tồn tại các bản ghi trùng lặp dựa trên cặp khóa (userId, movieId) hay không.

Quá trình kiểm tra được thực hiện bằng thư viện Pandas trong Python, sử dụng hàm duplicated() để phát hiện các bản ghi trùng lặp.

```
────────────────────────────────────────
3.3 KIỂM TRA TRÙNG LẶP (userId + movieId)
────────────────────────────────────────
Không có bản ghi trùng lặp
```

Kết quả kiểm tra cho thấy: Không tồn tại bản ghi trùng lặp trong bảng Ratings.

5. Kiểm tra tính hợp lệ của UserId, MovieId:

   Trong dataset MovieLens, mỗi bản ghi trong bảng Ratings bao gồm hai thuộc tính quan trọng:

   - userId: định danh của người dùng
   - movieId: định danh của bộ phim

   Theo cấu trúc của dataset, các giá trị này phải là số nguyên dương. Do đó, cần kiểm tra xem có tồn tại các giá trị userId hoặc movieId nhỏ hơn hoặc bằng 0 hay không.

   Quá trình kiểm tra được thực hiện bằng Python với thư viện Pandas, bằng cách lọc các bản ghi có userId ≤ 0 hoặc movieId ≤ 0.

```
────────────────────────────────────────
  3.4 KIỂM TRA userId VÀ movieId
────────────────────────────────────────
   userId: tất cả > 0
   movieId: tất cả > 0
```

   Kết quả kiểm tra cho thấy:

   - Tất cả các giá trị userId đều lớn hơn 0.
   - Tất cả các giá trị movieId đều lớn hơn 0.

6. Kiểm tra tính nhất quán dữ liệu:
   Tương tự như userId, mỗi bản ghi trong bảng Ratings cũng chứa một giá trị movieId để xác định bộ phim được đánh giá.

   Do đó, tất cả movieId trong bảng Ratings phải tồn tại trong bảng Movies. Nếu tồn tại movieId không có trong bảng Movies thì đó là dữ liệu không hợp lệ.

```
  3.5 KIỂM TRA NHẤT QUÁN userId
────────────────────────────────────────
   Tất cả userId trong ratings đều có trong u.user
────────────────────────────────────────
```

   Kết quả kiểm tra cho thấy:

- Tất cả movieId trong bảng Ratings đều tồn tại trong bảng Movies.

Như vậy, dataset không có bản ghi đánh giá nào tham chiếu đến bộ phim không tồn tại.

### Tổng kết:

Kết quả cho thấy dataset không tồn tại vấn đề nghiêm trọng về chất lượng dữ liệu. Do đó, dữ liệu được xác nhận đã sạch và sẵn sàng cho bước phân tích dữ liệu khám phá (EDA) cũng như xây dựng ma trận user–item cho hệ thống gợi ý.

### Phụ lục

Yêu cầu:

```
pip install pandas numpy matplotlib seaborn tabulate

# Import

import os

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.ticker as ticker

import seaborn as sns

from tabulate import tabulate


# CẤU HÌNH


DATA_DIR = "ml-100k"

OUTPUT_DIR = "output_eda"


os.makedirs(OUTPUT_DIR, exist_ok=True)

plt.rcParams["font.size"] = 11

sns.set_style("whitegrid")
```

```
SEP = "=" * 62
SEP2 = "-" * 62
```

## Đọc dữ liệu

```
# 1.  ĐỌC DỮ LIỆU TỪ CÁC FILE THỰC TẾ
print(SEP)
print("1: ĐỌC DỮ LIỆU")
print(SEP)

# --- u.data: ratings ---
# Cấu trúc: user_id \t item_id \t rating \t timestamp
df_ratings = pd.read_csv(
os.path.join(DATA_DIR, "u.data"),
sep="\t",
header=None,
names=["userId", "movieId", "rating", "timestamp"],
engine="python"
)

# --- u.item: thông tin phim ---
# Cấu trúc: movie_id | title | release_date | video_date | imdb_url | genre_1 ...
genre_19
genre_names = [
"unknown","Action","Adventure","Animation","Childrens","Comedy",
"Crime","Documentary","Drama","Fantasy","Film_Noir","Horror",
"Musical","Mystery","Romance","Sci_Fi","Thriller","War","Western"
]
item_cols = ["movieId","title","release_date","video_release_date","imdb_url"] +
genre_names
df_movies = pd.read_csv(
```

```
os.path.join(DATA_DIR, "u.item"),
sep="|",
header=None,
names=item_cols,
encoding="latin-1",
engine="python"
)

# --- u.user: thông tin người dùng ---
# Cấu trúc: user_id | age | gender | occupation | zip_code
df_users = pd.read_csv(
os.path.join(DATA_DIR, "u.user"),
sep="|",
header=None,
names=["userId","age","gender","occupation","zip_code"],
engine="python"
)

# --- u.genre: danh sách thể loại ---
df_genre = pd.read_csv(
os.path.join(DATA_DIR, "u.genre"),
sep="|",
header=None,
names=["genre","genre_id"],
engine="python"
)

# --- u.occupation: danh sách nghề nghiệp ---
df_occupation = pd.read_csv(
```

```
os.path.join(DATA_DIR, "u.occupation"),
header=None,
names=["occupation"],
engine="python"
)

print(f"Đọc thành công các file:")
print(f" u.data → {len(df_ratings):>7,} dòng (ratings)")
print(f" u.item → {len(df_movies):>7,} dòng (movies)")
print(f" u.user → {len(df_users):>7,} dòng (users)")
print(f" u.genre → {len(df_genre):>7,} dòng (thể loại)")
print(f" u.occupation → {len(df_occupation):>7,} dòng (nghề nghiệp)")

# 2: THỐNG KÊ TỔNG QUAN
print(f"\n{SEP}")
print("2: THỐNG KÊ TỔNG QUAN")
print(SEP)

n_users = df_ratings["userId"].nunique()
n_movies = df_ratings["movieId"].nunique()
n_ratings = len(df_ratings)
total_cells = n_users * n_movies
sparsity = 1 - (n_ratings / total_cells)
rating_mean = df_ratings["rating"].mean()
rating_std = df_ratings["rating"].std()
rating_med = df_ratings["rating"].median()

print(f"\n{'Chỉ số':<35} {'Giá trị':>15}")
print(SEP2)
```

```
print(f"{'Số lượng Users':<35} {n_users:>15,}")
print(f"{'Số lượng Movies':<35} {n_movies:>15,}")
print(f"{'Tổng số Ratings':<35} {n_ratings:>15,}")
print(f"{'Rating Min':<35} {df_ratings['rating'].min():>15}")
print(f"{'Rating Max':<35} {df_ratings['rating'].max():>15}")
print(f"{'Rating Trung bình (Mean)':<35} {rating_mean:>15.4f}")
```

## 2. Kiểm tra chất lượng dữ liệu

```
# 3: KIỂM TRA CHẤT LƯỢNG DỮ LIỆU (DATA QUALITY)
print(f"\n{SEP}")
print(" BƯỚC 3: KIỂM TRA CHẤT LƯỢNG DỮ LIỆU")
print(SEP)

issues_found = 0

# 3.1 Missing Values
print(f"\n{'─'*40}")
print(" 3.1 MISSING VALUES")
print(f"{'─'*40}")

for name, df in [("Ratings (u.data)", df_ratings),
("Movies (u.item)", df_movies[["movieId","title","release_date"]]),
("Users (u.user)", df_users)]:
mv = df.isnull().sum()
total_mv = mv.sum()
status = "Không có" if total_mv == 0 else f"CÓ {total_mv} missing"
print(f"\n [{name}] → {status}")
if total_mv > 0:
print(mv[mv > 0].to_string())
issues_found += total_mv
```

```
# 3.2 Giá trị Rating hợp lệ
print(f"\n{'─'*40}")
print(" 3.2 KIỂM TRA RATING HỢP LỆ [1–5]")
print(f"{'─'*40}")

invalid_ratings = df_ratings[~df_ratings["rating"].isin([1,2,3,4,5])]
print(f" Giá trị rating duy nhất: {sorted(df_ratings['rating'].unique())}")
if len(invalid_ratings) == 0:
print("Tất cả rating nằm trong khoảng [1, 5]")
else:
print(f"Có {len(invalid_ratings)} rating bất thường:")
print(invalid_ratings)
issues_found += len(invalid_ratings)

# 3.3 Trùng lặp
print(f"\n{'─'*40}")
print(" 3.3 KIỂM TRA TRÙNG LẶP (userId + movieId)")
print(f"{'─'*40}")

dups = df_ratings.duplicated(subset=["userId","movieId"])
n_dups = dups.sum()
if n_dups == 0:
print("Không có bản ghi trùng lặp")
else:
print(f"Có {n_dups} bản ghi trùng lặp")
print(df_ratings[dups].head(5))
issues_found += n_dups
```

```
# 3.4 userId & movieId hợp lệ
print(f"\n{'─'*40}")
print(" 3.4 KIỂM TRA userId VÀ movieId")
print(f"{'─'*40}")

invalid_user = df_ratings[df_ratings["userId"] <= 0]
invalid_movie = df_ratings[df_ratings["movieId"] <= 0]

if len(invalid_user) == 0:
print("userId: tất cả > 0")
else:
print(f"Có {len(invalid_user)} userId không hợp lệ")
issues_found += len(invalid_user)

if len(invalid_movie) == 0:
print("movieId: tất cả > 0")
else:
print(f"Có {len(invalid_movie)} movieId không hợp lệ")
issues_found += len(invalid_movie)

# 3.5 Kiểm tra tính nhất quán userId
print(f"\n{'─'*40}")
print(" 3.5 KIỂM TRA NHẤT QUÁN userId")
print(f"{'─'*40}")

users_in_ratings = set(df_ratings["userId"].unique())
users_in_uuser = set(df_users["userId"].unique())
orphan_users = users_in_ratings - users_in_uuser # có rating nhưng không có trong
u.user
```

```
if len(orphan_users) == 0:
print("Tất cả userId trong ratings đều có trong u.user")
else:
print(f"Có {len(orphan_users)} userId trong ratings không có trong u.user")
issues_found += len(orphan_users)

# 3.6 Kiểm tra tính nhất quán movieId
print(f"\n{'─'*40}")
print(" 3.6 KIỂM TRA NHẤT QUÁN movieId")
print(f"{'─'*40}")

movies_in_ratings = set(df_ratings["movieId"].unique())
movies_in_uitem = set(df_movies["movieId"].unique())
orphan_movies = movies_in_ratings - movies_in_uitem

if len(orphan_movies) == 0:
print("Tất cả movieId trong ratings đều có trong u.item")
else:
print(f"Có {len(orphan_movies)} movieId trong ratings không có trong u.item")
issues_found += len(orphan_movies)

# Tổng kết kiểm tra
print(f"\n{SEP}")
if issues_found == 0:
print("KẾT QUẢ: DỮ LIỆU SẠCH — Không phát hiện vấn đề nào!")
else:
print(f"KẾT QUẢ: Phát hiện {issues_found} vấn đề cần xử lý")
print(SEP)
```